



UNIVERSITÉ
CAEN
NORMANDIE

Projet 2 : Analysis of Sorting Algorithms

Mehmet Tuna Ozkalkanli
Andrea Gjoreska
Mila Bucevska

2026

Central Question

Objectives

Task distribution

EntropyGenerator

- Disorder vs Uncertainty
- Displacement
- Additional Details

Experimental Setup

- Random Input
- Sorted Input
- Low Entropy Input
- Medium Entropy Input
- High Entropy Input
- Reverse Sorted Input
- Reverse Low Entropy Input
- Reverse Medium Entropy Input
- Reverse High Entropy Input

Conclusion

Central Question

Question : How does the efficiency of each algorithm, based on different criteria, such as comparisons, swaps, data accesses, and execution time, vary depending on the quantity and distribution of disorder in the dataset input?

- Create a data generator with different disorder levels
- Implement the sorting algorithms with different behaviors
- Build a real-time visualization interface
- Track comparisons, swaps, and data accesses
- Analyze performance through benchmarking and compare the results

- **Mehmet Tuna Ozkalkanli**

- ▶ Data generators
- ▶ QuickSort, BogoSort
- ▶ Benchmark and CSV export
- ▶ Analysis

- **Andrea Gjoreska**

- ▶ Global structure and MVC architecture
- ▶ Visualization and compare mode
- ▶ InsertionSort, MergeSort, CocktailShakerSort, PancakeSort

- **Mila Bucevska**

- ▶ Event tracking and interface
- ▶ BubbleSort, BucketSort, CombSort
- ▶ Statistics window and graphs

Definition: Entropy measures how unpredictable the list structure is based on the proportion of displaced elements.

- Generates lists with a **target entropy level**
- Starts from a **sorted list**
- Introduces controlled **displacement**
- Keeps a valid **permutation**

Idea: Control *how disordered* a list is

- **Disorder (p):**
 - ▶ proportion of misplaced elements = (misplaced elements/list size)
- **Uncertainty ($H(p)$):**
 - ▶ how unpredictable the structure is

Examples:

- $[0, 0, 0, 0, 0] \rightarrow p = 0 \rightarrow$ fully sorted $\rightarrow$ low entropy
- $[1, 1, 1, 1, 1] \rightarrow p = 1 \rightarrow$ fully displaced $\rightarrow$ also low entropy
- both are **certain (predictable)** cases
- $p = 0.5 \rightarrow$ maximum entropy

1. Choosing the number of displaced elements

$$H(p) = -p \log_2(p) - (1 - p) \log_2(1 - p)$$

- $p = \frac{d}{n}$
- Try $d \in [0, n/2]$
- Choose d minimizing:

$$|H(p) - \text{target}|$$

Symmetry: $H(p) = H(1 - p)$

- If target > 0.5 , return $n - d$ to break symmetry and avoid generating identical lists

2. Applying the displacement

- Select d random indices
- Apply a cyclic permutation

Example:

$$[0, 1, 2, 3, 4, 5], \quad \text{indices} = [4, 1, 5]$$

$$4 \rightarrow 1. \quad 1 \rightarrow 5. \quad 5 \rightarrow 4$$

- Target entropy must be in $[0, 1]$
- Skip $d = 1$ (impossible case)
- Store:
 - ▶ `lastDisplaced`
 - ▶ `lastEntropy`
- Actual entropy $\neq$ target (approximation)

Key takeaway:

Generating data with **controlled "randomness"**

- **Algorithms tested:** BubbleSort, CocktailShakerSort, InsertionSort, MergeSort, QuickSort, CombSort, PancakeSort
- **Input sizes:** $n \in \{100, 500, 1000\}$ - each configuration repeated 4 times to reduce randomness
- **Generator types:**
 - ▶ Random
 - ▶ Entropy
 - ▶ Reverse entropy
- **Metrics collected:** number of comparisons, swaps, memory accesses, execution time

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 8.700	~ 2	$O(n \log n)$ - avg case
QuickSort	~ 11.250	~ 3	$O(n \log n)$ - avg case
CombSort	~ 22.700	~ 4	$O(n \log n)^*$ - avg case
InsertionSort	~ 252.000	~ 72	$O(n^2)$ - avg case
CocktailShaker	~ 377.000	~ 82	$O(n^2)$ - avg case
BubbleSort	~ 498.000	~ 110	$O(n^2)$ - avg case
PancakeSort	501.498	~ 129	$O(n^2)$ - avg case

Key Insight

On random input, $O(n \log n)$ algorithms perform up to **50× fewer comparisons** than quadratic ones. CombSort is the exception - a BubbleSort variant that scales surprisingly well in practice.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 5.044	~ 1	$O(n \log n)$ - stable
CombSort	~ 18.713	~ 2	$O(n \log n)^*$ - stable
BubbleSort	999	~ 0	$O(n^2)$ - best case
CocktailShaker	999	~ 0	$O(n^2)$ - best case
InsertionSort	999	~ 0	$O(n^2)$ - best case
QuickSort	~ 499.500	~ 138	$O(n^2)$ - worst case
PancakeSort	501.498	~ 54	$O(n^2)$ - insensitive

Key Insight

Sorted input **inverts** the hierarchy. Adaptive algorithms drop to $n - 1$ comparisons, but QuickSort degrades to $O(n^2)$ - its worst case - due to poor pivot selection on already sorted data.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 6.950	~ 2	$O(n \log n)$ - stable
CombSort	~ 21.700	~ 3	$O(n \log n)^*$ - stable
InsertionSort	~ 15.100	~ 4	$O(n^2)$ - near best case
CocktailShaker	~ 22.800	~ 5	$O(n^2)$ - near best case
BubbleSort	~ 466.000	~ 58	$O(n^2)$ - degrading
QuickSort	~ 63.650	~ 10	$O(n^2)$ - unstable
PancakeSort	501.498	~ 64	$O(n^2)$ - insensitive

Key Insight

Nearly sorted input reveals **adaptivity as a real advantage**. InsertionSort becomes competitive with MergeSort despite being $O(n^2)$ - partial order drastically reduces its work. QuickSort remains unpredictable due to pivot sensitivity.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 7.700	~ 1	$O(n \log n)$ - stable
QuickSort	~ 33.000	~ 4	$O(n \log n)$ - recovering
CombSort	~ 22.700	~ 3	$O(n \log n)^*$ - stable
InsertionSort	~ 37.000	~ 8	$O(n^2)$ - losing edge
CocktailShaker	~ 60.500	~ 13	$O(n^2)$ - losing edge
BubbleSort	~ 494.000	~ 100	$O(n^2)$ - avg case
PancakeSort	501.498	~ 87	$O(n^2)$ - insensitive

Key Insight

At medium entropy, adaptive algorithms **lose their structural advantage** and converge toward average-case behavior. QuickSort recovers significantly, while InsertionSort and CocktailShaker fade back into their quadratic nature.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 8.700	~ 2	$O(n \log n)$ - stable
QuickSort	~ 11.500	~ 2	$O(n \log n)$ - avg case recovered
CombSort	~ 22.700	~ 3	$O(n \log n)^*$ - efficient
InsertionSort	~ 217.000	~ 70	$O(n^2)$ - adaptivity fading
CocktailShaker	~ 320.000	~ 80	$O(n^2)$ - quadratic again
BubbleSort	~ 499.000	~ 100	$O(n^2)$ - average case
PancakeSort	501.498	~ 129	$O(n^2)$ - not adaptive

Key Insight

At high entropy, most of the initial order disappears. **QuickSort** fully recovers and behaves close to its expected average case, while **InsertionSort** and **CocktailShaker** lose most of the structural advantage they had on nearly sorted input.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 4.932	~ 1	$O(n \log n)$ - stable
CombSort	~ 19.700	~ 1	$O(n \log n)^*$ - stable
QuickSort	~ 499.500	~ 100	$O(n^2)$ - worst case
InsertionSort	~ 499.500	~ 135	$O(n^2)$ - worst case
BubbleSort	~ 499.500	~ 140	$O(n^2)$ - worst case
CocktailShaker	~ 500.000	~ 140	$O(n^2)$ - worst case
PancakeSort	501.498	~ 70	$O(n^2)$ - not adaptive

Key Insight

Reverse-sorted input is the **mirror worst case**. Local-swap algorithms collapse to full quadratic cost, and **QuickSort** also degenerates because the last-element pivot creates maximally unbalanced partitions. Only **MergeSort** and **CombSort** remain truly robust.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~6.950	~2	$O(n \log n)$ - stable
CombSort	~21.700	~2	$O(n \log n)^*$ - stable
QuickSort	~53.000	~10	$O(n \log n)$ - unstable recovery
InsertionSort	~487.000	~140	$O(n^2)$ - near worst case
CocktailShaker	~499.750	~130	$O(n^2)$ - near worst case
BubbleSort	~499.500	~150	$O(n^2)$ - near worst case
PancakeSort	501.498	~84	$O(n^2)$ - not adaptive

Key Insight

A small amount of reverse disorder is **not enough to break the reverse structure**. BubbleSort, CocktailShaker, and InsertionSort remain near worst case. QuickSort improves, but it is still unstable compared with **MergeSort** and **CombSort**.

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~7.700	~2	$O(n \log n)$ - stable
CombSort	~22.700	~3	$O(n \log n)^*$ - stable
QuickSort	~23.000	~4	$O(n \log n)$ - recovering strongly
InsertionSort	~466.000	~118	$O(n^2)$ - still near worst case
CocktailShaker	~499.000	~135	$O(n^2)$ - still penalized
BubbleSort	~499.500	~128	$O(n^2)$ - still penalized
PancakeSort	501.498	~76	$O(n^2)$ - not adaptive

Key Insight

At medium reverse entropy, the reverse bias is still present, but it is no longer dominant for every algorithm. **QuickSort** recovers strongly toward average-case behavior, while **BubbleSort**, **CocktailShaker**, and **InsertionSort** remain trapped in near worst-case

Algorithm	Comparisons	Time (ms)	Complexity
MergeSort	~ 8.700	~ 2	$O(n \log n)$ - stable
QuickSort	~ 12.000	~ 2	$O(n \log n)$ - avg case recovered
CombSort	~ 22.700	~ 3	$O(n \log n)^*$ - stable
InsertionSort	~ 290.000	~ 61	$O(n^2)$ - still expensive
CocktailShaker	~ 430.000	~ 98	$O(n^2)$ - still quadratic
BubbleSort	~ 499.500	~ 118	$O(n^2)$ - still quadratic
PancakeSort	501.498	~ 116	$O(n^2)$ - not adaptive

Key Insight

Once reverse entropy becomes high, the initial reverse order is mostly broken. **QuickSort** becomes efficient again and joins **MergeSort** and **CombSort** among the best performers. In contrast, the quadratic algorithms still scale poorly despite the additional disorder.

- The experiments confirm that **input structure strongly influences performance**
- Three behaviors emerge:
 - ▶ **Strongly structure-sensitive:** BubbleSort, CocktailShakerSort, InsertionSort
 - ▶ **Mostly stable across inputs:** MergeSort
 - ▶ **Intermediate / partially sensitive:** CombSort, QuickSort
- Adaptive algorithms perform extremely well on low entropy input, but degrade quickly toward $O(n^2)$ as disorder increases
- In contrast, $O(n \log n)$ algorithms remain **stable and predictable** regardless of input distribution

- **MergeSort** is the most consistent, fast and reliable algorithm
 - ▶ Always close to $O(n \log n)$
 - ▶ Unaffected by entropy or input order
- **QuickSort** can be the fastest (beating MergeSort with 1-2 ms in some cases) in practice on random/high-entropy data
 - ▶ Efficient on average
 - ▶ But unstable due to pivot choice (worst-case degradation)
- **CombSort** is a clear improvement over BubbleSort
 - ▶ Same idea but with gaps
 - ▶ Much fewer comparisons in practice
 - ▶ Scales closer to $O(n \log n)$ behavior **experimentally**

- **Nearly sorted data:**
 - ▶ Best choice: **InsertionSort**
- **Random or high entropy data:**
 - ▶ Best choice for speed in lower list sizes: **QuickSort**
 - ▶ Best choice for overall stability: **MergeSort**
- **Unknown input:**
 - ▶ Best choice: **MergeSort**
- **Simple improvement over BubbleSort:**
 - ▶ Use **CombSort**

Overall Winner

MergeSort is the overall winner.

- It consistently performs **50x fewer comparisons** than quadratic algorithms on large inputs (e.g., $\sim 8,700$ vs $\sim 500,000$ at $n = 1000$)
- Execution time remains extremely low across all configurations
- Unlike QuickSort, it does not suffer from worst-case degradation
- Therefore, it is the most predictable and reliable algorithm overall